Hard Top-K' with an annealed active count

Research note: linear and log-scale K-annealing

2026-10-27.

The K-annealing schedule

Setup. The model and training are those of the code-carried hard baselines of the note `docs/kj-vs-hard-topk-code-residual.tex`:
8 blocks of width 768 on TinyStories, an abs-TopK bottleneck over $N = 1536$ features after every block with the sparse code carried from block to block (`code_residual`), RMSNorm on each bottleneck output, $S = 20000$ steps with the cosine learning-rate schedule, seed 1337. A hard Top-K' bottleneck keeps the K' features of largest magnitude and passes the gradient to those K' only.

Annealing. At optimizer step $s = 0, \ldots, S-1$ every bottleneck is an ordinary hard Top-$K'(s)$ bottleneck, forward and backward, with the width falling from $K + J$ to K (here $K = 32$ and $J = 480$, 224 or 96):

$$K'(s) = K + J - \min\big(J,\ \lfloor (J+1)\, f(s) \rfloor\big), \qquad f(0) = 0, \quad f(S) = 1,$$

where $f(s)$ is the fraction of the J extra features removed by step s. The *linear* schedule has $f(s) = s/S$: one feature every $S/(J+1) \approx 42$ steps. The *log* schedule removes features at a rate proportional to $1/(s + s_0)$,

$$f(s) = \frac{\ln(1 + s/s_0)}{\ln(1 + S/s_0)}, \qquad s_0 = \frac{h^2}{S - 2h},$$

where $h \in (0, S/2)$ is the step at which half of the J features have been removed, $f(h) = 1/2$. Small h removes most features early; as $h \to S/2$ the log schedule tends to the linear one. At $J = 480$ the grid is $h \in \{125, 250, 500, 1000, 2000, 4000, 6000, 8000\}$ ($s_0 = 0.79$ to 16000). At $J = 224$ and $J = 96$, starting from $K' = 256$ and $K' = 128$, it is $h \in \{30, 60, 120, 250\}$ ($s_0 = 0.045$ to 3.2). The width K' at step s is a function of s only, identical at all eight bottlenecks.

Figures

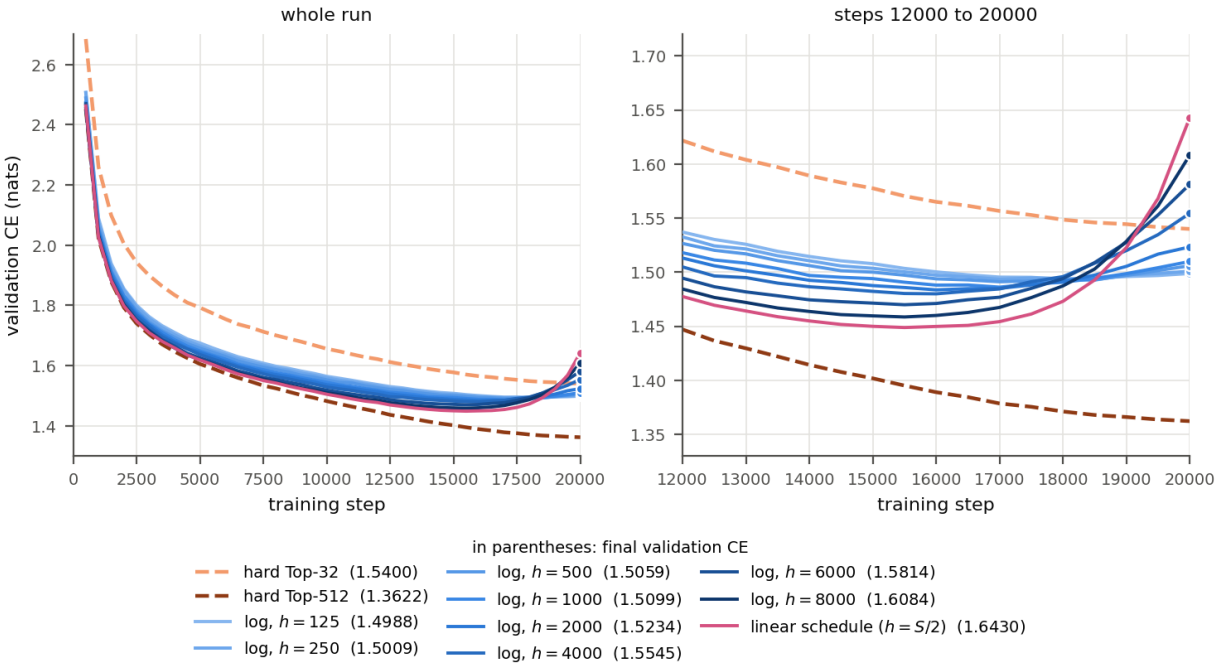


Figure 1: Validation cross-entropy (the same 960 windows of 512 tokens for every run, every 500 steps). Dashed: hard Top-32 (light orange) and hard Top-512 (dark orange) at fixed width. Solid: K-annealing from $K' = 512$ to $K = 32$: the log schedule in blue, from light ($h = 125$) to dark ($h = 8000$), and the linear schedule ($h = S/2$) in magenta. The validation loss of an annealed run at step s is measured at its current width $K'(s)$ (Figure 2); only its last point (dot, step 20000) is at $K = 32$ and comparable to hard Top-32. Left: the whole run; right: steps 12000 to 20000. Legend: final validation CE.

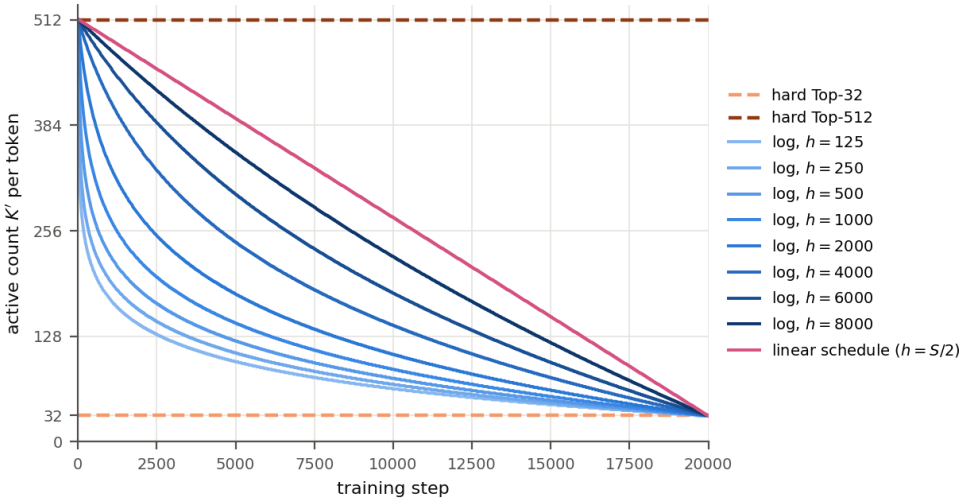


Figure 2: Active count K' per token at every bottleneck, as logged by each run (`bottleneck/active_count`, every 20 steps). Colors as in Figure 1; the dashed lines are the fixed widths of the hard baselines.

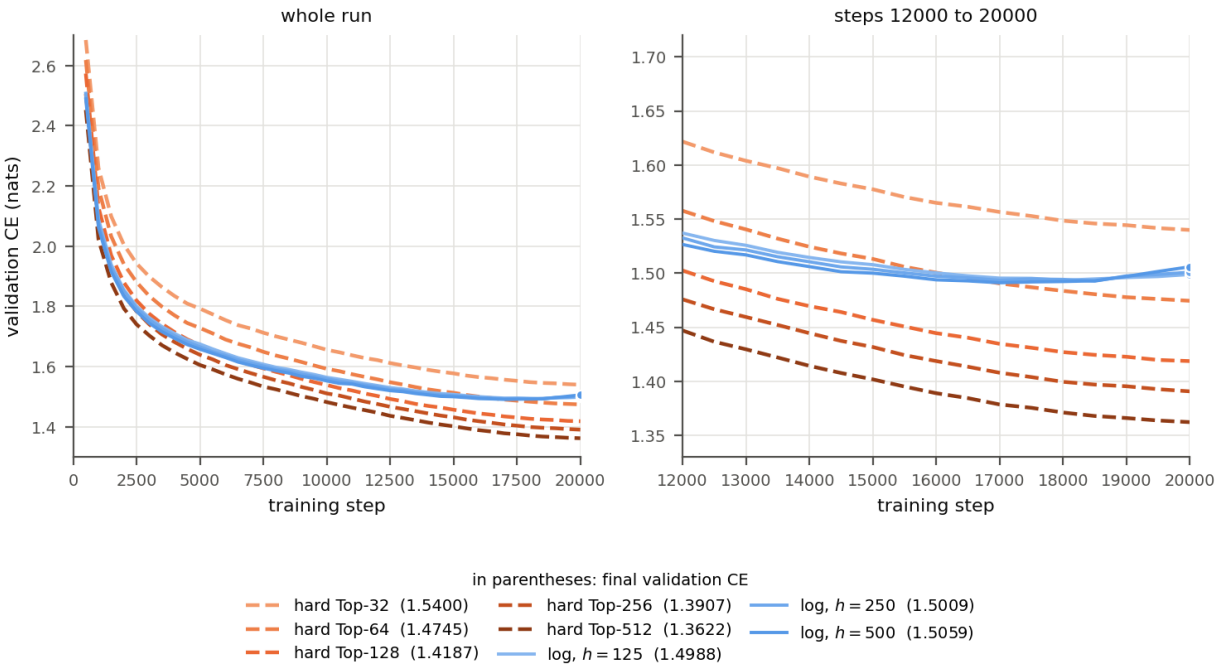


Figure 3: Validation cross-entropy of the log schedule at $h = 125, 250, 500$ (solid, blue as in Figure 1) against the code-carried hard Top-K ladder $K = 32, 64, 128, 256, 512$ of `docs/kj-vs-hard-topk-code-residual.tex` (dashed, orange from light to dark with K). The annealed runs are measured at their current width $K'(s)$; only their last point (dot) is at $K = 32$. Left: the whole run; right: steps 12000 to 20000. Legend: final validation CE.

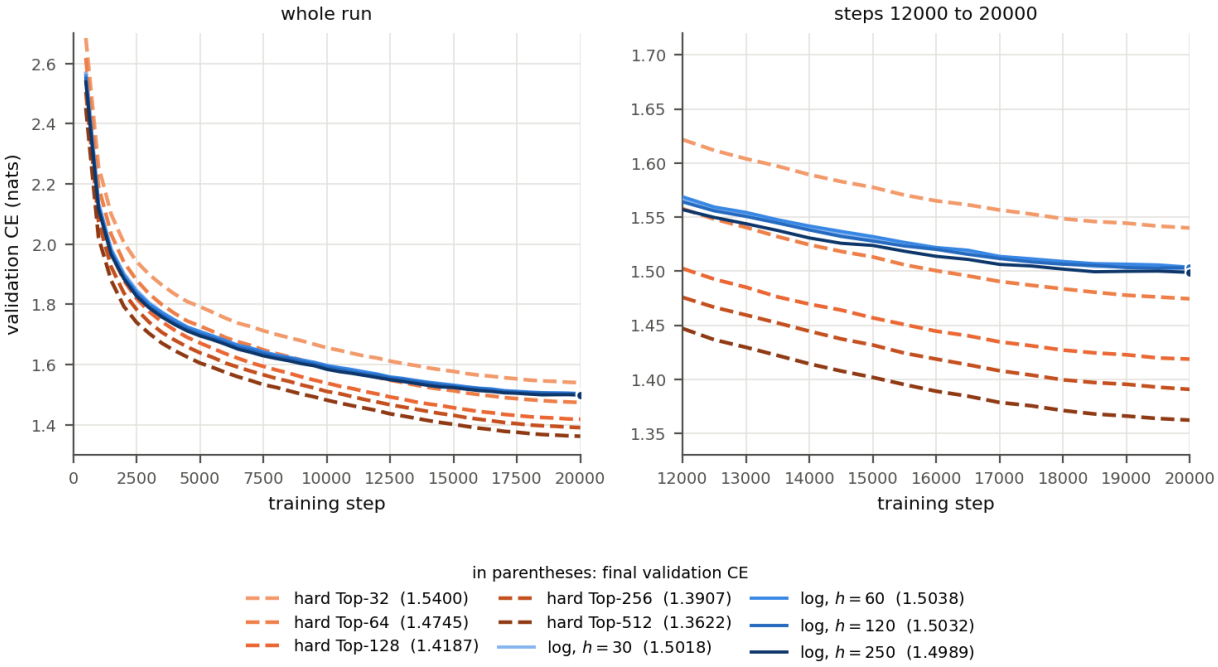


Figure 4: As Figure 3 at $J = 224$: the log schedule from $K' = K + J = 256$ to $K = 32$ at $h = 30$, 60, 120, 250 (solid, blue from light to dark with h) against the code-carried hard Top-K ladder (dashed, orange from light to dark with K). The annealed runs are measured at their current width $K'(s)$; only their last point (dot) is at $K = 32$. Left: the whole run; right: steps 12000 to 20000. Legend: final validation CE.

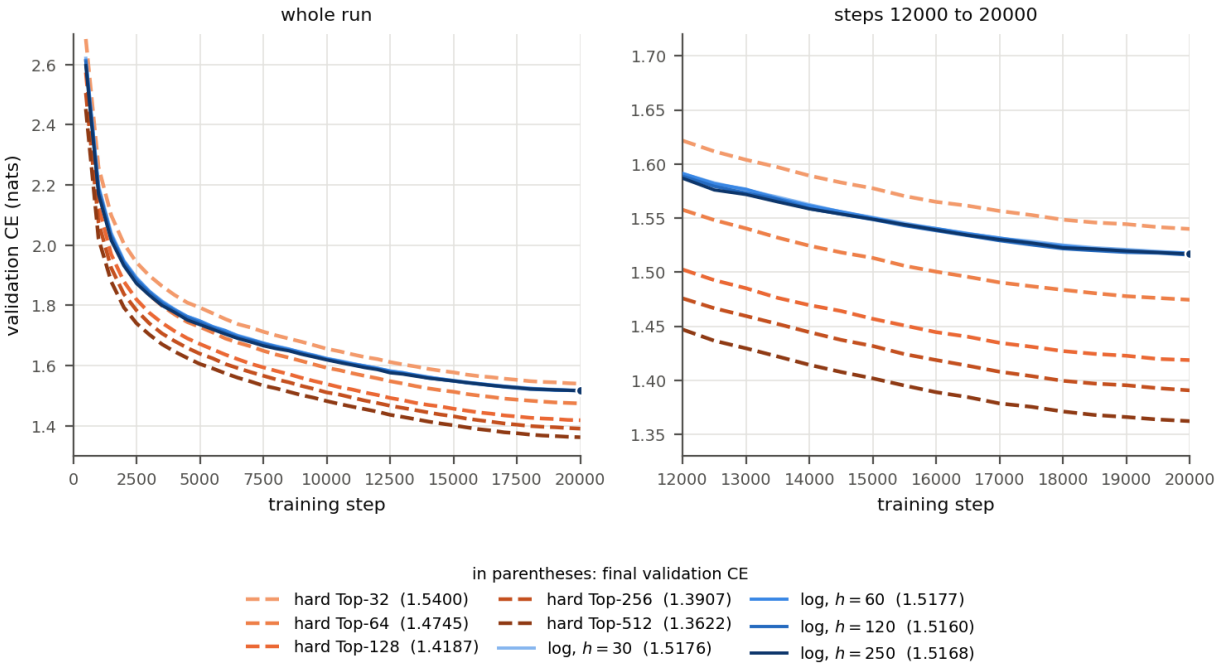


Figure 5: As Figure 3 at $J = 96$: the log schedule from $K' = K + J = 128$ to $K = 32$ at $h = 30$, 60, 120, 250 (solid, blue from light to dark with h) against the code-carried hard Top-K ladder (dashed, orange from light to dark with K). The annealed runs are measured at their current width $K'(s)$; only their last point (dot) is at $K = 32$. Left: the whole run; right: steps 12000 to 20000. Legend: final validation CE.

Runs

All under `gdrive:weight-sparsity/`; configs in the repository.

curve	Drive	config
hard Top-32	`runs_madrid/ma_cr_hard_k32_pnorm`	`configs/cr_kj/`
hard Top-64	`runs_madrid/ma_cr_hard_k64_pnorm`	`configs/cr_kj/`
hard Top-128	`runs_madrid/ma_cr_hard_k128_pnorm`	`configs/cr_kj/`
hard Top-256	`runs_madrid/ma_cr_hard_k256_pnorm`	`configs/cr_kj/`
hard Top-512	`runs_madrid/ma_cr_hard_k512_pnorm`	`configs/cr_kj/`
linear, $J = 480$	`runs_riga/ri_cr_hard_kanneal_k32_j480_pnorm`	`configs/hard_k_anneal/`
log, $J = 480$, $h = 125$	`runs_tallinn/ta_cr_hard_klog_h125_k32_j480_pnorm`	`configs/hard_k_anneal/`
log, $J = 480$, $h = 250$	`runs_tallinn/ta_cr_hard_klog_h250_k32_j480_pnorm`	`configs/hard_k_anneal/`
log, $J = 480$, $h = 500$	`runs_tallinn/ta_cr_hard_klog_h500_k32_j480_pnorm`	`configs/hard_k_anneal/`
log, $J = 480$, $h = 1000$	`runs_tallinn/ta_cr_hard_klog_h1000_k32_j480_pnorm`	`configs/hard_k_anneal/`
log, $J = 480$, $h = 2000$	`runs_tallinn/ta_cr_hard_klog_h2000_k32_j480_pnorm`	`configs/hard_k_anneal/`
log, $J = 480$, $h = 4000$	`runs_tallinn/ta_cr_hard_klog_h4000_k32_j480_pnorm`	`configs/hard_k_anneal/`
log, $J = 480$, $h = 6000$	`runs_tallinn/ta_cr_hard_klog_h6000_k32_j480_pnorm`	`configs/hard_k_anneal/`
log, $J = 480$, $h = 8000$	`runs_tallinn/ta_cr_hard_klog_h8000_k32_j480_pnorm`	`configs/hard_k_anneal/`
log, $J = 224$, $h = 30$	`runs_tallinn/ta_cr_hard_klog_h30_k32_j224_pnorm`	`configs/hard_k_anneal/`
log, $J = 224$, $h = 60$	`runs_tallinn/ta_cr_hard_klog_h60_k32_j224_pnorm`	`configs/hard_k_anneal/`
log, $J = 224$, $h = 120$	`runs_tallinn/ta_cr_hard_klog_h120_k32_j224_pnorm`	`configs/hard_k_anneal/`
log, $J = 224$, $h = 250$	`runs_tallinn/ta_cr_hard_klog_h250_k32_j224_pnorm`	`configs/hard_k_anneal/`
log, $J = 96$, $h = 30$	`runs_tallinn/ta_cr_hard_klog_h30_k32_j96_pnorm`	`configs/hard_k_anneal/`
log, $J = 96$, $h = 60$	`runs_tallinn/ta_cr_hard_klog_h60_k32_j96_pnorm`	`configs/hard_k_anneal/`
log, $J = 96$, $h = 120$	`runs_tallinn/ta_cr_hard_klog_h120_k32_j96_pnorm`	`configs/hard_k_anneal/`
log, $J = 96$, $h = 250$	`runs_tallinn/ta_cr_hard_klog_h250_k32_j96_pnorm`	`configs/hard_k_anneal/`

Each run directory holds `config.json`, `metrics.jsonl`, `summary.json`, the TensorBoard events (`tb/`) and the checkpoints every 2000 steps. The figures are drawn by `scripts/plot_kanneal.py` from `docs/kanneal/data/curves.json`, extracted from the runs' `metrics.jsonl`.